🟢 Aula 8 — Consumo de API com Axios (4h)

🎯 Objetivos da aula

- Entender o funcionamento de **REST APIs**.
- Diferenciar **Promises** e `async/await`.
- Consumir APIs externas usando **Axios**.
- Implementar **loading**, **tratamento de erros** e **boas práticas**.
- Integrar **API + Vue Router + Componentes**.

---

1) Teoria (aprox. 1h30)

1.1 O que é uma REST API

Uma **REST API** expõe dados e ações via **HTTP**:

Método	Ação
GET	Buscar dados
POST	Criar
PUT	Atualizar
DELETE	Remover

Exemplo:

```
GET https://api.exemplo.com/users
```

Resposta (JSON):

```
[ { "id": 1, "name": "Ana" }, { "id": 2, "name": "Bruno" } ]
```

1.2 Promises vs async/await

Promises (forma tradicional)

```
axios.get(url) .then(res => console.log(res.data)) .catch(err => console.error(err))
```

---

async/await (recomendado)

```
try { const res = await axios.get(url) console.log(res.data) } catch (error) { console.error(error) }
```

✔️ Código mais legível
✔️ Fluxo semelhante a código síncrono

1.3 Tratamento de erros (boas práticas)

- `try/catch` para falhas de rede
- Mensagens claras para o usuário
- Evitar expor erro técnico bruto
- Sempre controlar estado de **loading**

---

2) Prática (aprox. 2h30) — Consumo de API pública

🎯 **Contexto**
Consumir a API pública **JSONPlaceholder** para:

- Buscar usuários remotamente
- Exibir lista dinâmica
- Mostrar loading e erro

API:

```
https://jsonplaceholder.typicode.com/users
```

2.1 Instalação do Axios

```
npm install axios
```

2.2 Estrutura usada

```
src/
 ├─ views/
 │   └─ UsersView.vue
 ├─ services/
 │   └─ api.js
 ├─ App.vue
 └─ main.js
```

---

3) Centralizando acesso à API (boa prática)

3.1 `src/services/api.js`

```
import axios from 'axios' // Cria uma instância do Axios const api = axios.create({ baseURL:
'https://jsonplaceholder.typicode.com', timeout: 5000 // evita requisições travadas }) // Interceptor simples (opcional)
api.interceptors.response.use( response => response, error => { console.error('Erro na API:', error) return Promise.reject(error)
} ) export default api
```

---

4) `UsersView.vue` — Listagem remota com loading e erro

```
<template> <section> <h2>Usuários (API Externa)</h2> <!-- Loading --> <p v-if="loading">Carregando usuários...</p> <!-- Erro -->
<p v-else-if="erro" class="error"> {{ erro }} </p> <!-- Lista --> <ul v-else> <li v-for="user in usuarios" :key="user.id">
<strong>{{ user.name }}</strong> — {{ user.email }} </li> </ul> </section> </template> <script> import api from '../services/api'
export default { name: 'UsersView', data() { return { usuarios: [], loading: false, erro: '' } }, async mounted() { // Hook ideal
para carregar dados visuais this.loading = true try { const response = await api.get('/users') this.usuarios = response.data }
catch (error) { this.erro = 'Erro ao carregar usuários. Tente novamente.' } finally { this.loading = false } } } </script> <style
scoped> .error { color: #c62828; } </style>
```

---

5) O que foi aplicado (teoria → prática)

✔ **REST API**

- Consumo via HTTP
- Dados JSON

✔ **async/await**

```
const response = await api.get('/users')
```

✔️ Tratamento de erro

```
try { ... } catch { ... } finally { ... }
```

✔️ Estados de UI

- `loading`
- `erro`
- `dados`

---

6) Atividade prática da aula

🎯 Desafio

1. Adicionar botão “**Recarregar**”
2. Extrair a lógica de carregamento para um método `carregarUsuarios`
3. Simular erro:
   - Alterar a URL da API temporariamente
4. Mostrar mensagem personalizada para erro 404

💡 Dica:

```
if (error.response?.status === 404) { this.erro = 'Recurso não encontrado.' }
```

7) Checklist de encerramento

- Instalou Axios
- Usou async/await
- Controlou loading
- Tratou erros corretamente
- Separou camada de API